

Classical Modeling and Modern Methods

Annotated outline for a 15-minute Text Mining lecture segment

Mary Kate Carara | Working document for PowerPoint development

Opening and speaking map

Opening bridge from Lauren: Preprocessing converts article text into usable tokens and features. The next question is what a model should learn from them. A classifier predicts a label supplied by people; a topic model discovers recurring themes; embeddings represent meaning in vectors; and a pretrained language model can supply contextual representations for several tasks. The same self-driving-car news setting connects all four methods.

Example policy: Every article sentence, class label, topic, and vector relationship below is illustrative. No probabilities, learned topics, embedding distances, or performance claims are reported from the project dataset.

Segment	Minutes	Speaking goal
Opening and transitions	0.5	Connect representation to modeling.
Logistic regression	3.0	Explain supervised classification.
LDA overview	1.5	Introduce unsupervised topics.
Word and document embeddings	4.5	Move from counts to dense vectors.
Pretrained language models	4.5	Explain contextual reuse with BERT.
Comparison and handoff	1.0	Summarize and pass LDA to Laura.
Total	15.0	

1 Logistic regression for classification

3 minutes

Sources

- Qamar and Raza (2024), chapters on feature engineering and text classification: the text-mining pipeline and its supervised tasks.
- Weiss et al. (2005), chapters on numerical text vectors and prediction: why document text must become features before prediction.
- scikit-learn developers (2026), logistic regression section: probability model and classification threshold.

Key concepts and goals

- **Goal:** predict a predefined class for a new article using examples whose classes are already known.
- **Inputs:** a document feature vector such as word counts or TF-IDF weights; **output:** a class probability and, after a threshold, a label.
- The feature vector is sparse and high-dimensional; the model learns a weight for each feature. A weight describes association with the chosen label, not a causal effect.

Method

Given labeled training articles, represent each article by a feature vector $\mathbf{x}$. Logistic regression combines the features linearly and maps the score to a probability:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp[-(b + \mathbf{w}^\top \mathbf{x})]}.$$

Here $y = 1$ is the class of interest, b is an intercept, and $\mathbf{w}$ contains learned feature weights. A chosen threshold converts the probability to a label; 0.5 is a simple illustration, not a required choice. Training needs labeled articles, and prediction uses the same feature construction as training (scikit-learn developers 2026; Qamar and Raza 2024). This explanation assumes Lauren has already introduced bag-of-words and TF-IDF.

Figures

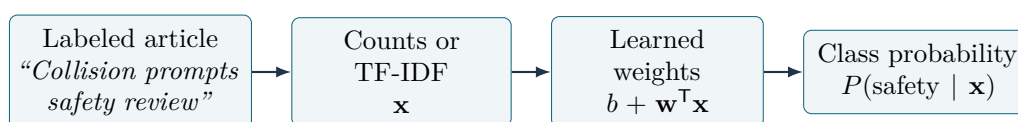


Figure 1: Original teaching schematic of text classification, based on the feature-to-prediction descriptions of Weiss et al. (2005) and scikit-learn developers (2026). No values shown are fitted results.

Example

Illustrative only. Suppose a class defines *safety incident* versus *policy*. A headline about an autonomous-vehicle collision might receive the safety label, while one about a bill governing road tests might receive the policy label. A story about a bill introduced after a collision could contain both themes; the binary task still requires an explicit labeling rule. Do not present these labels or any predicted probability as results from the group’s articles.

Key takeaways

- Logistic regression answers a supervised question: *Which known label fits this article?*
- It provides an interpretable baseline, but its bag-of-words or TF-IDF input does not itself represent word order or contextual meaning.

2 LDA: a brief classical overview

1.5 minutes

Sources

- Blei et al. (2003), the original Latent Dirichlet Allocation paper: documents as mixtures of topics and topics as distributions over words.
- Silge and Robinson (2017), Chapter 6: an accessible two-topic example and interpretation of topic-word and document-topic probabilities.
- Qamar and Raza (2024), chapter on text summarization and topic modeling: connection to a classical text-mining workflow.

Key concepts and goals

- **Goal:** find recurring themes in a collection of articles without supplying topic labels for each article.
- A *topic* is a distribution over words; an *article* can contain a mixture of topics. The analyst interprets and names topics after fitting (Blei et al. 2003; Silge and Robinson 2017).
- LDA here means *Latent Dirichlet Allocation*. It is a different method from the supervised classifier just introduced.

Method

Start with a document-term representation of the article collection. Choose the number of topics, fit the model, then inspect prominent words for each topic and the topic mixture for each article. The model estimates these distributions together; it does not receive a prewritten list of “safety” or “policy” topic names (Blei et al. 2003; Silge and Robinson 2017). Leave the generative process, inference, and topic evaluation to Laura’s dedicated LDA segment.

Figures

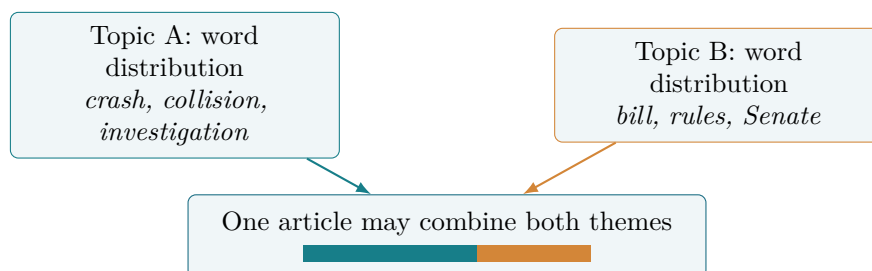


Figure 2: Original schematic of topic-word distributions and an article-topic mixture, based on Blei et al. (2003) and Silge and Robinson (2017). The words and bar lengths are illustrative, not fitted topics or probabilities.

Example

Illustrative only. An article about a proposed testing law following an autonomous-vehicle crash could use words associated with both a safety theme and a regulation theme. LDA could assign it a mixture of those discovered topics; this is not a manually assigned class or a claim about the project data.

Key takeaways

- LDA answers an unsupervised question: *What themes recur across these articles?*
- Its output is interpretable topic-word and article-topic distributions. **Handoff to Laura:** how LDA estimates and evaluates them.

3 Word and document embeddings

4.5 minutes

Sources

- Mikolov et al. (2013), Figure 1: word2vec's CBOW and Skip-gram architectures.
- Le and Mikolov (2014), Figure 2: learned paragraph or document vectors.
- Reimers and Gurevych (2019): later contextual sentence embeddings.

Key concepts and goals

- Bag-of-words vectors have many zero vocabulary entries; learned embeddings are shorter, dense vectors (Mikolov et al. 2013; Le and Mikolov 2014).
- **Goals:** give related words useful nearby vectors and give each article one vector for comparison or downstream learning.
- Classic word2vec vectors are *static*: each word has the same vector in different sentences.

Method

CBOW predicts a center word from its neighbors; Skip-gram predicts neighbors from a center word. Both learn vectors for word types (Mikolov et al. 2013). Paragraph Vector learns a document vector that joins local context in word prediction and represents text of varying length (Le and Mikolov 2014). The resulting vectors can be compared or used as model features. This approach differs from extracting embeddings with a pretrained language model.

Figures

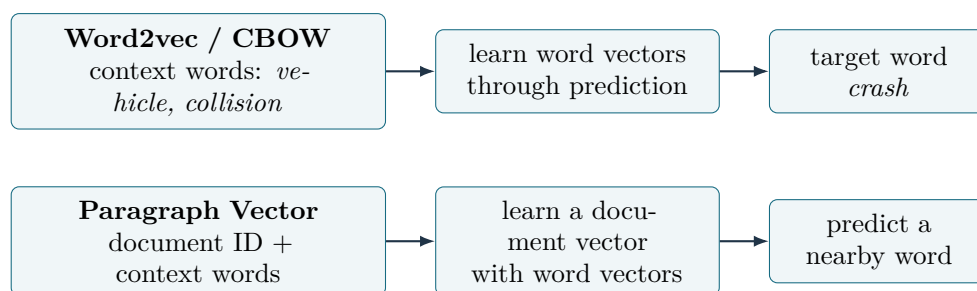


Figure 3: Simplified adaptations of Figure 1 in Mikolov et al. (2013) (original paper) and Figure 2 in Le and Mikolov (2014) (original paper). Example words are illustrative.

Example

Illustrative only. “Autonomous-vehicle collision triggers inquiry” and “Driverless-car crash prompts investigation” share few exact words, yet a suitable learned representation could place them near each other. Actual closeness depends on the model; no distance was measured.

Key takeaways

- Embeddings turn words or entire documents into vectors that can support similarity, clustering, or classification.
- Earlier word2vec-style word vectors do not change with context; contextual embeddings from pretrained models can.

4 Pretrained language models

4.5 minutes

Sources

- Devlin et al. (2019), especially Figure 1: BERT pretraining followed by task-specific fine-tuning.
- Reimers and Gurevych (2019): Sentence-BERT’s fixed-size embeddings for semantic similarity and clustering.

Key concepts and goals

- **Pretraining:** learn general patterns from large amounts of text before the particular class task. **Reuse:** adapt the model or its representations to a later task (Devlin et al. 2019).
- BERT is a Transformer encoder that uses surrounding context on both sides of a token; its masked-token prediction is the pretraining idea to explain aloud (Devlin et al. 2019).
- The same written word can receive a different contextual representation in different sentences. Task choice matters: a BERT classifier and a model trained to produce comparable sentence vectors have different outputs (Devlin et al. 2019; Reimers and Gurevych 2019).

Method

Present a two-stage workflow: pretrain BERT on general text, then add a task head and fine-tune it with labeled articles for classification. For similarity or clustering, use a sentence-embedding model such as Sentence-BERT that is trained to produce comparable fixed-size vectors; do not assume an arbitrary raw BERT output is already a good similarity embedding (Devlin et al. 2019; Reimers and Gurevych 2019). Mention, in one sentence, that generative language models are another pretrained family that commonly learns to predict the next token; the segment does not cover their architecture or generation behavior.

Figures

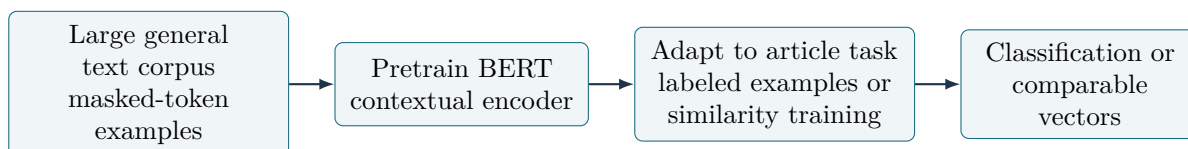


Figure 4: Simplified adaptation of the pretraining-to-task flow in Figure 1 of Devlin et al. (2019) (original paper); the similarity-output branch is informed by Reimers and Gurevych (2019) (paper).

Example

Illustrative only. Compare “The company *charged* the autonomous car overnight” with “The company was *charged* after the autonomous-car crash.” The surrounding words give *charged* different meanings; BERT can produce representations conditioned on each sentence. For the group’s news corpus, a task-specific classifier might predict a chosen article label, while a sentence-embedding model could support article similarity or clustering. Neither behavior is claimed as a measured result on the corpus.

Key takeaways

- Pretraining supplies reusable language representations that can be adapted with less task-specific data than training a comparable model from scratch.
- Choose an output suited to the goal: class probabilities for classification, comparable document vectors for similarity or clustering.

Comparison and handoff

1 minute

Method	Main question	Main output
Logistic regression	Which supplied label?	Class probability / label
LDA	What themes recur?	Topic-word and article-topic distributions
Embeddings	Which words or articles are similar?	Dense vectors
Pretrained language model	How can general language learning help this task?	Contextual features, labels, or task-trained vectors

Closing line: “We have seen what LDA contributes alongside classification and embeddings: interpretable themes without article labels. Laura will now show how LDA finds those themes and how we can judge them.”

References

- Blei, David M., Andrew Y. Ng, and Michael I. Jordan (2003). “Latent Dirichlet Allocation”. In: *Journal of Machine Learning Research* 3, pp. 993–1022. URL: <https://jmlr.csail.mit.edu/papers/v3/blei03a.html>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186. DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423). URL: <https://aclanthology.org/N19-1423/>.
- Le, Quoc and Tomas Mikolov (2014). “Distributed Representations of Sentences and Documents”. In: *Proceedings of the 31st International Conference on Machine Learning*. Vol. 32. Proceedings of Machine Learning Research, pp. 1188–1196. URL: <https://proceedings.mlr.press/v32/le14.html>.
- Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean (2013). “Efficient Estimation of Word Representations in Vector Space”. In: *International Conference on Learning Representations*. URL: <https://research.google/pubs/efficient-estimation-of-word-representations-in-vector-space/>.
- Qamar, Usman and Muhammad Summair Raza (2024). *Applied Text Mining*. Springer Nature Switzerland. DOI: [10.1007/978-3-031-51917-8](https://doi.org/10.1007/978-3-031-51917-8).
- Reimers, Nils and Iryna Gurevych (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 3982–3992. DOI: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410). URL: <https://aclanthology.org/D19-1410/>.
- scikit-learn developers (2026). *1.1. Linear Models: Logistic Regression*. URL: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression (visited on 10/09/2026).
- Silge, Julia and David Robinson (2017). *Text Mining with R: A Tidy Approach*. Chapter 6: Topic Modeling. O’Reilly Media. URL: <https://www.tidytextmining.com/topicmodeling.html> (visited on 10/09/2026).
- Weiss, Sholom M., Nitin Indurkha, Tong Zhang, and Fred J. Damerau (2005). *Text Mining: Predictive Methods for Analyzing Unstructured Information*. Springer. DOI: [10.1007/978-0-387-34555-0](https://doi.org/10.1007/978-0-387-34555-0).